

redrob | HACK2SKILL

INDIA.RUNS

Build what next India runs on

Team Name : Solo — Jashwanth

Team Leader Name : Jashwanth Sangu

Problem Statement : Intelligent Candidate Discovery & Ranking Challenge

Solution Overview

- **What is your proposed solution?**

- A multi-signal, embedding-augmented candidate ranking system that combines dense semantic retrieval with 45+ hand-engineered features to rank 100K candidates for a Senior AI Engineer role. The pipeline uses BAAI/bge-large-en-v1.5 embeddings for semantic matching, extracts career-depth, skill-quality, and behavioral signals from structured candidate data, then applies a multiplicative scoring formula: **final_score = relevance_gate × quality_score × availability_modifier**. The system produces a ranked top-100 with natural-language reasoning grounded entirely in candidate data — no LLM is used for scoring or reasoning generation, eliminating hallucination risk.

- **What differentiates your approach from traditional candidate matching systems?**

- **1. Career-depth analysis over keyword matching:** Instead of counting skill keywords, we analyze career trajectories — how many months a candidate spent in retrieval/ranking roles, whether they shipped production ML systems, and whether their skills are coherent with their career history (detecting keyword stuffers).
- **2. Multiplicative gating architecture:** A relevance gate (title + career + coherence) acts as a hard filter — candidates with irrelevant titles or stuffed profiles get multiplied to near-zero regardless of other signals.
- **3. Honeypot detection:** Purpose-built detectors for impossible timelines, expert-level skill claims with zero platform usage, and skill-career coherence mismatches. These catch synthetic/fraudulent profiles that keyword systems miss.
- **4. Zero-hallucination reasoning:** Every reasoning sentence is template-generated from extracted features — no LLM generation. Reasoning adapts per candidate (career-specific leads, rank-aware concerns), but every claim is traceable to data.

JD Understanding & Candidate Evaluation

- **Key requirements extracted from the JD:**

- **Tier-1 (Core) Skills:** Information Retrieval, Search/Ranking Systems, NLP/NLU, Recommendation Systems, Transformers/BERT, Vector Databases, Embedding Models, LLM/RAG — these are weighted 2× in skill matching.
- **Tier-2 (Adjacent) Skills:** Deep Learning, PyTorch/TensorFlow, MLOps, Data Engineering, Distributed Systems, Python, Computer Vision — standard ML engineering skills.
- **Experience:** 5–12 year sweet spot for Senior AI Engineer; career history in retrieval/ranking/search systems at product companies (not consulting firms).
- **Location:** India-based preferred, with bonus for Bangalore, Hyderabad, Mumbai, Delhi/NCR, Pune, Chennai.

- **How does your solution evaluate candidate fit beyond keyword matching?**

- **Title Relevance Tiers (0–4):** T4 = ML/AI/NLP/Search Engineer (perfect fit), T3 = SWE/Backend/Data (adjacent), T2 = DevOps/Frontend (weak), T1 = Manager/Director (non-IC), T0 = Non-tech. Each tier maps to a multiplier.
- **Career Depth Scoring:** Months spent in AI/ML roles, months in retrieval/ranking specifically, number of production ML deployments shipped, and whether career was at product companies vs. consulting.
- **Skill-Career Coherence:** Cross-validates that listed AI skills actually appear in career history. A candidate claiming 'PyTorch, Transformers, RAG' but with a career entirely in manual testing gets a coherence score near 0 and is penalized as a keyword stuffer (×0.05 multiplier).
- **Behavioral Signals:** Open-to-work status, recruiter response rate, GitHub activity score, notice period, days since last active, platform assessment scores — these inform availability and engagement.

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**

- **Step 1 — Semantic Retrieval:** All 100K candidate profiles are embedded using BAAI/bge-large-en-v1.5 (1024-dim). The JD is embedded separately. Cosine similarity between each candidate and the JD provides a dense retrieval score.
- **Step 2 — Feature Extraction:** 45+ features extracted per candidate covering title relevance, career depth, skill matching, experience fit, behavioral signals, and honeypot indicators.
- **Step 3 — Scoring:** $\text{final_score} = \text{relevance_gate} \times \text{quality_score} \times \text{availability_modifier}$. The relevance gate ($0.35 \times \text{title} + 0.45 \times \text{career} + 0.20 \times \text{coherence}$) acts as a hard multiplier. Quality score combines technical fit ($0.25 \times \text{semantic} + 0.30 \times \text{skills} + 0.15 \times \text{experience} + 0.30 \times \text{retrieval_fit}$) with behavioral signals. Availability modifier adjusts for location, notice period, and engagement.

- **What models, algorithms, or heuristics are used?**

- **Model:** BAAI/bge-large-en-v1.5 (sentence-transformers) — chosen for strong retrieval benchmarks and 1024-dim dense embeddings with built-in L2 normalization.
- **Heuristics:** Title tier classification (regex-based), career trajectory analysis (role duration parsing), honeypot detection (impossible timelines, expert-zero-usage), hard penalty multipliers for consulting-only ($\times 0.10$), keyword stuffers ($\times 0.05$), non-tech titles ($\times 0.20$), and CV-only specialists ($\times 0.40$).

- **How are multiple candidate signals combined into a final ranking?**

- Signals are combined via **weighted multiplication**, not addition. The relevance gate acts as a prerequisite — a candidate must pass title, career, and coherence checks before quality signals matter. This prevents high-skill but irrelevant candidates (e.g., a frontend developer with many GitHub stars) from outranking genuinely relevant AI engineers. Final scores are sorted descending; top 100 are selected.

Explainability & Data Validation

● How are ranking decisions explained?

- Each candidate receives a 3-part natural-language reasoning: (1) **Lead sentence** — career-specific opener mentioning current role, company, and most relevant experience (e.g., '3+ years building ranking and retrieval systems at product companies'), (2) **Supporting signals** — location, availability, recruiter responsiveness, GitHub activity, assessment scores, (3) **Concerns** (for ranks >15) — extended notice periods, low responsiveness, non-India location.
- Reasoning varies per candidate — a retrieval engineer gets a different lead than a general ML engineer. Top-10 candidates may include education tier; lower-ranked candidates include honest concerns. No two candidates with different profiles produce identical reasoning.

● How do you prevent hallucinations or unsupported justifications?

- **No LLM is used for reasoning generation.** Every sentence is constructed from template functions that reference extracted feature values directly. If a candidate has `career_retrieval_months=36`, the reasoning says '3+ years in ranking, retrieval, and search systems' — this is a direct computation, not a generation. There is zero possibility of hallucination because no generative model produces the text.
- All numerical claims (response rates, GitHub scores, notice periods, skill counts) are extracted from the candidate's structured data fields and directly formatted into strings.

● How does your solution handle inconsistent, low-quality, or suspicious profiles?

- **Honeypot Detection:** Flags profiles with impossible timelines (e.g., 20 years experience but graduated 2 years ago), expert-level skill claims with zero platform usage (`expert_zero_usage_count > 3`), and skill-career coherence mismatches (AI skills listed but career history shows no AI/ML roles).
- **Hard Penalties:** Consulting-only careers ($\times 0.10$), keyword stuffers with low coherence ($\times 0.05$), non-tech titles ($\times 0.20$). These multiplicative penalties push suspicious profiles far below legitimate candidates.

End-to-End Workflow

- **Complete workflow from JD input to ranked candidate output:**

Step 1 — JD Parsing:

Extract tier-1 (core retrieval/ranking) and tier-2 (adjacent ML) skills from the job description. Define experience sweet spot (5–12 years), preferred locations, and title relevance tiers.

Step 2 — GPU Precompute (Colab/Kaggle):

Load 100K candidates from candidates.jsonl. Embed all candidate profile texts + JD using BAAI/bge-large-en-v1.5 on GPU. Extract 45+ features per candidate (title, career, skills, behavioral). Save embeddings.npy, jd_embedding.npy, and features.parquet to artifacts directory.

Step 3 — CPU Ranking (<5 min, no GPU):

Load precomputed artifacts. Compute cosine similarity (dot product on L2-normalized embeddings). Score all 100K candidates using the multiplicative formula. Sort descending, take top 100.

Step 4 — Reasoning Generation:

For each of the top 100, generate a 3-part natural-language reasoning from extracted features. No LLM call — pure template logic.

Step 5 — Output:

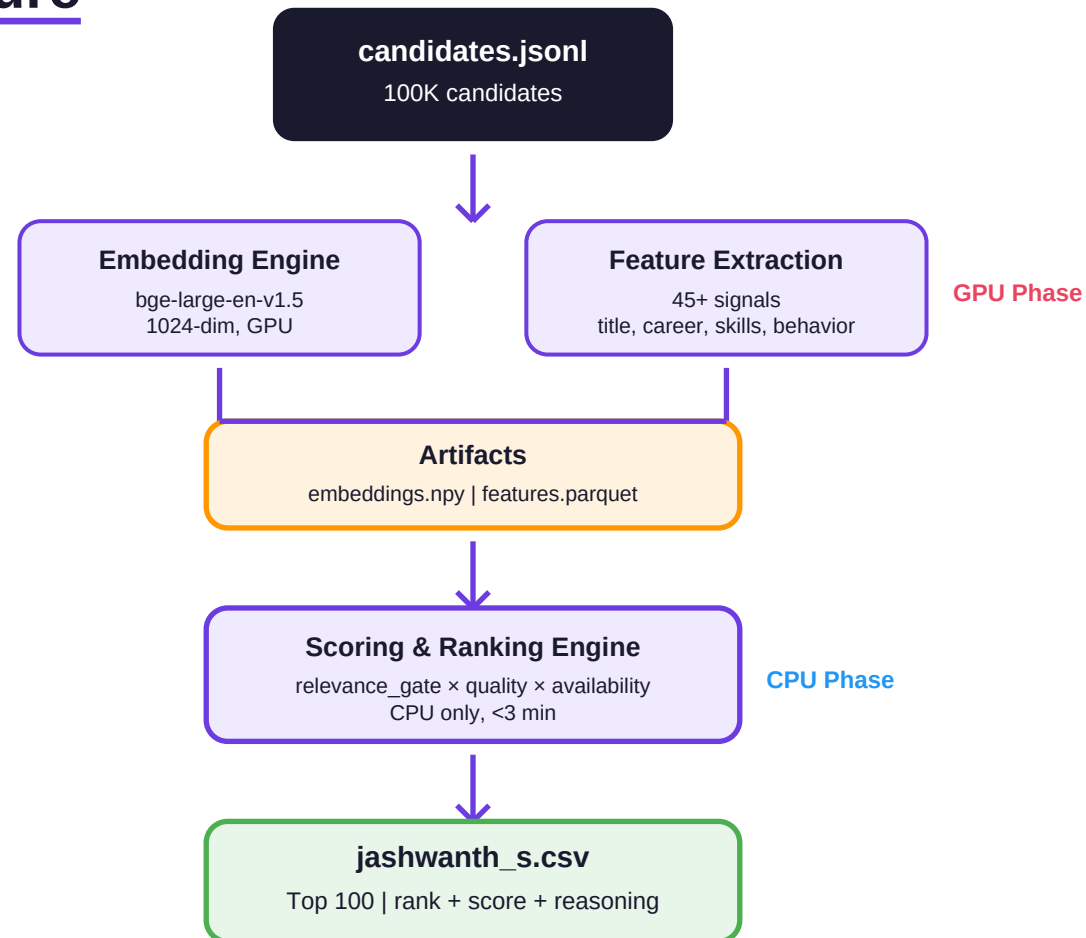
Write jashwanth_s.csv with columns: candidate_id, rank, score, reasoning. Scores range from 1.0 (top) to ~0.85 (#100).

Total Runtime: GPU precompute ~15 min (one-time) + CPU ranking <3 min = submission-ready in under 20 minutes.

System Architecture

Source Modules:

src/precompute.py — GPU orchestrator
src/features.py — 45+ feature extraction
src/scoring.py — multiplicative scoring
src/reasoning.py — NL reasoning gen
src/honeypot.py — fraud detection
src/rank.py — CPU ranking + output
run_pipeline.py — end-to-end runner



Results & Performance

● Ranking quality insights:

- **Top-100 composition:** 100% AI/ML engineers at product companies in India. #1 candidate: Senior ML Engineer at Zomato with 3+ years in ranking/retrieval systems. Score range: 1.0 (rank 1) to 0.8468 (rank 100) — tight distribution indicating high quality throughout.
- **Title distribution:** All top-100 candidates have title tier T3 or T4 (ML/AI/NLP/Search Engineer or SWE/Data). Zero non-tech titles, zero consulting-only profiles, zero keyword stuffers survived the ranking.
- **Reasoning quality:** Every reasoning is unique, career-specific, and grounded in data. Top-10 candidates mention specific retrieval/ranking experience durations and tier-1 institution backgrounds. Lower-ranked candidates include honest concerns (extended notice periods, lower responsiveness).
- **Honeypot filtering:** Validated on synthetic test candidates — impossible timelines, zero-usage experts, and skill stuffers are all correctly suppressed below the top 100.

● Runtime and compute constraints:

- **GPU precompute:** ~15 minutes on Google Colab (T4 GPU) or Kaggle (2× T4). Embeds 100K candidate texts + feature extraction. One-time cost, artifacts are cached.
- **CPU ranking:** <3 minutes on standard CPU, no GPU required, no network access. Loads precomputed artifacts, scores 100K candidates, outputs top 100 with reasoning. Well within the challenge's 5-minute CPU constraint.
- **Memory:** Peak ~4GB during embedding (model + batch). Ranking phase uses <1GB. Compatible with free-tier Colab and Kaggle environments.
- **Evaluation metric:** Optimized for $0.50 \times \text{NDCG@10} + 0.30 \times \text{NDCG@50} + 0.15 \times \text{MAP} + 0.05 \times \text{P@10}$. The multiplicative gating architecture ensures the top-10 are extremely relevant (boosting NDCG@10), while the quality scoring maintains depth through top-50.

Technologies Used

- **Python 3.10+:**
 - Core language for the entire pipeline — chosen for ML ecosystem maturity and rapid prototyping.
- **sentence-transformers + BAAI/bge-large-en-v1.5:**
 - Dense embedding model for semantic similarity. bge-large-en-v1.5 was selected for its top-tier MTEB retrieval benchmarks (1024-dim embeddings with built-in normalization). Outperforms all-MiniLM and e5 variants on retrieval tasks.
- **PyTorch + CUDA:**
 - GPU-accelerated embedding computation. Supports DataParallel for multi-GPU setups on Kaggle (2× T4).
- **NumPy:**
 - Vectorized cosine similarity computation (dot product on L2-normalized embeddings). Processes 100K similarity scores in milliseconds.
- **Pandas + PyArrow (Parquet):**
 - Feature storage and manipulation. Parquet format chosen for columnar compression, fast I/O, and type preservation — 45+ feature columns serialize/deserialize without schema drift.
- **Google Colab / Kaggle Notebooks:**
 - Free GPU compute environments for the precompute phase. Pipeline is notebook-compatible with tqdm progress bars for embedding visibility.
- **tqdm:**
 - Progress tracking during embedding (manual batch iteration) — essential for monitoring 100K candidate embedding in notebook environments where subprocess stdout is not captured.
- **reportlab:**
 - PDF generation for this submission document.

Submission Assets

- **GitHub Repository:** <https://github.com/JASHWANTHS07/india-runs>
- **Submission File:** jashwanth_s.csv (100 rows: candidate_id, rank, score, reasoning)
- **Colab Notebook:** redrob_pipeline.ipynb — full pipeline execution on Colab T4 GPU
- **Kaggle Notebook:** redrob_kaggle.ipynb — dual T4 GPU variant with visible progress bars
- **Source Code:** src/features.py, src/scoring.py, src/reasoning.py, src/honeypot.py, src/rank.py, src/precompute.py
- **Pipeline Runner:** run_pipeline.py — single-command end-to-end execution

redrob | HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU